Triton AIOS 复刻方案（Beta 2.5）

文档版本：Beta 2.5　最后更新：2026年8月

> **变更说明**：Beta 2 在 v1.0 基础上整合两大核心机制——**动态表单返回机制**与**原子工具箱**，并据此重构通信协议、SDK 接口与架构关系图，**删除全部"预设 API"相关设计**。Beta 2.5 进一步补充三处细节：**8.2 节明确 handle 返回结构（steps 与视图并行）**、**8.2 节新增 `context.tool_results` 格式定义**、**6.3 节新增 actions 回传路由机制**。

目录

1. 概述
2. 整体架构
3. 桌面系统设计
4. 主控（调度器）设计
5. 应用（小 AI）设计
6. 通信协议（核心 · 重构）
7. 原子工具箱（核心 · 新增）
8. SDK 设计（核心 · 重构）
9. 系统控制应用
10. 语音交互设计
11. 系统启动流程
12. 用户交互流程
13. 应用开发模板
14. 系统配置文件
15. 开发路线图
16. 总结

一、概述

1.1 项目定位

Triton AIOS 是一个 AI Native 桌面操作系统原型。它的核心思想是：

- 用户通过自然语言与系统交互
- 系统由一个大模型驱动的主控和若干专业子 Agent（应用）组成
- 每个子 Agent 是一个独立的小 AI，拥有自己的 GUI 与执行能力
- 子 Agent 之间通过主控协调，互不干扰

- **系统不预设任何高级 API**：只提供原子工具，由小 AI 自主规划组合完成任务
- **系统不解析业务数据**：只负责按 `view_type` 渲染 UI、按 `actions` 生成交互控件、转发操作请求

1.2 设计原则

原则	说明
AI 只动嘴，代码只动手	所有 AI（主控和小 Agent）只输出 JSON 指令，真正的系统操作由代码执行器安全执行
应用即文件夹	每个应用是一个独立目录，包含自己的前后端代码与小 AI 提示词
进程级隔离	每个应用运行在独立进程中，互不影响
通信标准化	所有通信遵循统一的 JSON 协议（含 `view_type` / `actions` / `steps`）
GUI 由系统渲染	小 AI 返回 `view_type` + `data`，系统按视图类型渲染，应用不再自己写 HTML
原子工具，自由组合	系统只提供原子工具（HTTP、文件、命令等），不预设高级 API；小 AI 自己规划如何组合
数据不透明	系统不解析 `data` 内容，只负责展示与转发，业务语义完全由小 AI 决定
语音原生支持	语音输入与语音播报作为一等公民

1.3 与 v1.0 的关键差异

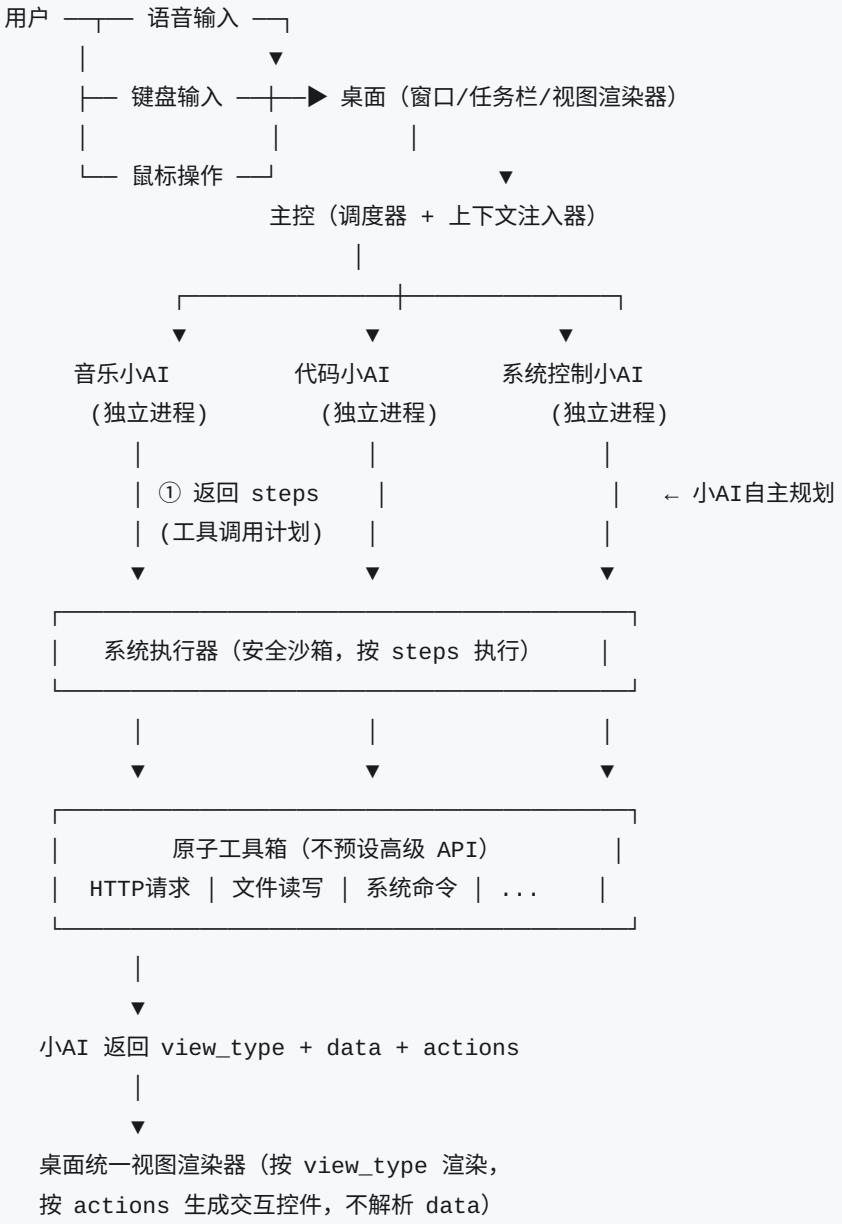
维度	v1.0（预设 API 模式）	Beta 2（原子工具模式）
应用能力声明	开发者在 `capabilities` 中硬编码 `action` / `params`	小 AI 运行时自主规划 `steps`，无预设能力清单
UI 渲染	应用 `render()` 返回完整 HTML	小 AI 返回 `view_type` + `data`，系统统一渲染
交互控件	应用自行实现按钮/表单	小 AI 通过 `actions` 声明，系统统一生成
工具调用	应用调用自己的 `/api/*` 路由	小 AI 调用系统提供的原子工具接口
环境感知	应用读环境变量	系统启动时为小 AI 注入上下文（角色/工作目录/状态）

二、整体架构

2.1 四层架构

层级	名称	职责
第 1 层	桌面层	窗口管理、任务栏、应用图标、系统托盘、**统一视图渲染器**
第 2 层	主控层	意图理解、任务拆解、Agent 调度、会话记忆、**上下文注入**
第 3 层	应用层	各专业小 AI（音乐、代码、系统控制等），独立进程，**自主规划工具组合**
第 4 层	工具层	**原子工具箱**：HTTP 请求、文件读写、系统命令等，不预设高级 API

2.2 核心模块关系



> **关键关系**：系统提供原子工具 → 小 AI 接收用户指令后自主规划 `steps` → 系统执行器安全执行 → 小 AI 拿到工具结果后返回 `view_type` + `data` + `actions` → 系统渲染。系统始终是"工具提供者 + 执行器 + 渲染器"，**不是**"API 预设者"。

2.3 数据流时序

```
用户指令
  │
  ▼
主控：意图理解 → 匹配到某个小AI
  │
  ▼
主控：为小AI 注入上下文（角色/工作目录/当前状态）
  │
  ▼
小AI：理解指令 → 规划 steps（工具调用计划）
  │
  ▼
系统执行器：按 steps 顺序调用原子工具箱，收集每个工具的返回
  │
  ▼
小AI：拿到工具结果 → 组织 view_type + data + actions
  │
  ▼
桌面渲染器：按 view_type 渲染视图，按 actions 生成控件，data 原样填充
  │
  ▼
用户看到界面，点击 actions 触发 → 操作请求转发回小AI → 循环
```

三、桌面系统设计

3.1 桌面 UI 构成

元素	说明
桌面背景	渐变或壁纸，所有窗口的底层容器
应用窗口	每个应用独立窗口，可拖拽、缩放、最小化、关闭
任务栏	固定在底部，包含开始菜单、应用图标、状态区、时钟
开始菜单	点击后展示所有已安装应用列表
系统托盘	显示系统状态（网络、音量、API 连接状态）
统一视图渲染器	根据 `view_type` 渲染小 AI 返回的视图，根据 `actions` 生成交互控件

3.2 窗口管理规范

功能	说明
创建窗口	点击应用图标 → 创建新窗口 → 由渲染器按 `view_type` 渲染内容
拖拽移动	拖拽标题栏 → 窗口跟随移动
窗口缩放	拖拽窗口边缘 → 改变窗口大小
最小化	点击最小化按钮 → 窗口收起至任务栏
最大化	点击最大化按钮 → 窗口铺满桌面（保留任务栏）
关闭	点击关闭按钮 → 销毁窗口
窗口切换	点击任务栏图标 → 对应窗口提到最前
Z-index 管理	点击窗口 → 自动提升层级

3.3 任务栏交互

交互	行为
点击应用图标	打开对应应用窗口
点击已打开的应用图标	将对应窗口提到最前
右键任务栏	弹出系统菜单（设置、刷新、关于）
点击时钟	展开日历（可选）

3.4 应用发现机制

系统启动时自动扫描 `apps/` 目录：

1. 遍历 `apps/` 下的所有子目录
2. 检查子目录中是否包含 `manifest.json`（应用清单）
3. 读取应用元信息（名称、图标、描述、入口文件、**小 AI 角色提示词**）
4. 为该应用分配一个空闲端口
5. 启动该应用的独立进程
6. 在任务栏中渲染应用图标
7. 将端口映射写入 `port_map.json`

应用清单格式（`manifest.json`）：

字段	类型	必填	说明
name	string	是	应用显示名称
icon	string	否	应用图标（emoji 或图标名）
description	string	否	应用功能描述
entry	string	是	入口文件（如 `main.py`）
role_prompt	string	是	**小 AI 角色提示词**（系统启动时注入上下文）
workdir	string	否	小 AI 的工作目录（默认 `apps/{应用名}/workspace`）
port	number	否	指定端口（不指定则自动分配）
author	string	否	作者信息
version	string	否	版本号

Beta 2 变更 新增 `role_prompt` 与 `workdir`。`role_prompt` 决定小 AI 的人格与能力边界，`workdir` 限定小 AI 的文件操作范围。**不再要求声明** `capabilities`。

四、主控（调度器）设计

4.1 主控职责

职责	说明
意图理解	将用户输入转化为结构化意图
任务拆解	将复合指令拆解为多步任务，生成 DAG
Agent 匹配	根据任务类型匹配合适的小 AI
上下文注入	系统启动时为小 AI 注入上下文（角色、工作目录、当前状态）
调度执行	按依赖顺序调用小 AI，管理执行流程
工具计划转发	将小 AI 返回的 `steps` 转交系统执行器，不自行解释
会话记忆	维护会话上下文，记录执行历史
结果汇总	收集各小 AI 返回的 `view_type` + `data` + `actions`，组织回复用户

4.2 主控工作流程

```
用户输入（文字/语音）
        ↓
【意图理解】调用大模型解析意图
        ↓
【任务拆解】生成任务列表（含依赖关系）
        ↓
【Agent匹配】为每个任务匹配对应小AI
        ↓
【上下文注入】为小AI注入角色/工作目录/当前状态
        ↓
【调度执行】小AI返回 steps → 系统执行器执行 → 结果回传小AI
        ↓
【结果汇总】收集 view_type + data + actions
        ↓
【回复用户】桌面渲染器统一渲染视图与交互控件
```

4.3 上下文注入机制（新增）

系统在调度小 AI 前，会为其注入一份上下文（Context），让小 AI 感知自身环境：

```
{
    "role": "你是一个音乐小AI，负责搜索和播放音乐",
    "workdir": "/apps/music_app/workspace",
    "cwd_state": {
        "files": ["recent.json", "playlist.m3u"],
        "last_played": "周杰伦 - 七里香"
    },
    "available_tools": ["http_request", "file_read", "file_write", "shell_exec"],
    "session_id": "sess_abc123",
    "history_summary": "用户刚问过周杰伦的歌"
}
```

小 AI 基于此上下文：

- 知道自己是谁（`role`）
- 知道在哪个目录工作（`workdir`）
- 知道当前环境状态（`cwd_state`）
- 知道有哪些原子工具可用（`available_tools`）
- 知道会话历史（`history_summary`）

4.4 会话记忆结构

字段	说明
session_id	会话唯一标识
history	对话历史列表
current_tasks	当前正在执行的任务列表
completed_tasks	已完成的任务列表
context	上下文变量（如当前播放歌曲、打开文件等）
agent_states	各小 AI 的暂存状态
tool_calls_log	原子工具调用历史（审计用）

五、应用（小 AI）设计

5.1 应用定义

一个应用 = 一个独立的小 AI 运行单元，拥有自己的：

- **角色提示词**（决定小 AI 的人格与能力边界）
- **工作目录**（小 AI 文件操作的沙箱根目录）
- **独立进程**（与其他应用隔离）
- **自主规划能力**（小 AI 接收用户指令后，自己决定如何组合原子工具）

> Beta 2 变更 应用不再是"全栈 Web 应用 + 自己写 HTML"。UI 由系统根据 `view_type` 统一渲染，应用只负责"思考 + 调用工具 + 返回数据"。

5.2 应用目录结构

```
apps/
└── {应用名}/
    ├── manifest.json      # 应用清单（含 role_prompt、workdir）
    ├── main.py            # 小AI 入口（加载角色、接收指令、返回结果）
    ├── role.md            # 小AI 角色提示词（可选，也可内联在 manifest）
    ├── workspace/         # 小AI 的工作目录（沙箱根）
    │    └── ...           # 小AI 通过原子工具在此读写
    └── requirements.txt   # 依赖声明
```

5.3 应用生命周期

阶段	触发	行为
安装	用户将应用文件夹放入 `apps/`	系统扫描并识别
启动	系统启动或用户点击图标	分配端口，启动进程，**注入上下文**
运行	用户与应用交互	小 AI 规划 steps → 执行器执行 → 返回视图
暂停	窗口最小化	进程仍在运行，UI 隐藏
停止	用户关闭窗口	销毁窗口，进程继续运行
卸载	用户删除应用文件夹	系统下次启动时不再加载

六、通信协议（核心 · Beta 2 重构）

6.1 协议总览

所有通信遵循统一的 JSON 协议。Beta 2 在 v1.0 基础上新增三个关键字段：

字段	类型	必填	说明
`view_type`	string	是	视图类型，系统据此选择渲染模板（共 10 种）
`data`	object	是	业务数据，结构完全自由，**系统不解析**，只原样填充到视图
`actions`	array	否	可操作元素定义（按钮、表单、下拉菜单等），系统据此生成交互控件
`steps`	array	否	工具调用计划，小 AI 规划后由系统执行器执行（共 10 种工具）
`status`	string	是	`ok` / `error`
`message`	string	否	给用户的自然语言说明

6.2 view_type：10 种视图类型

系统内置 10 种视图类型，小 AI 通过 `view_type` 指定渲染方式，`data` 提供数据：

view_type	用途	data 示例结构（自由，仅供参考）
`text`	纯文本展示	`{"content": "你好，世界"}`
`list`	列表展示	`{"items": [{"title": "七里香", "sub": "周杰伦"}]}`
`card`	卡片网格	`{"cards": [{"title": "...", "image": "...", "desc": "..."}]}`
`table`	表格展示	`{"columns": ["歌名", "歌手"], "rows": [["七里香", "周杰伦"]]}`
`form`	表单输入	`{"fields": [{"name": "keyword", "label": "关键词", "type": "text"}]}`
`media`	媒体播放（音/视频）	`{"url": "...", "type": "audio", "title": "七里香"}`
`image`	图片展示	`{"url": "...", "caption": "..."}`
`chart`	图表展示	`{"type": "bar", "labels": [...], "values": [...]}`
`code`	代码展示	`{"language": "python", "code": "print('hi')"}`
`dashboard`	综合仪表盘	`{"widgets": [{"type": "text", ...}, {"type": "chart", ...}]}`

> 系统不校验 `data` 是否符合某种 schema——`data` 是"黑盒"，系统只把它原样填入对应视图模板的插槽。

6.3 actions：10 种交互控件

`actions` 数组定义可操作元素，每个元素含 `type`、`label`、`on_trigger` 等字段。系统据此自动生成交互控件，用户触发后，系统将操作请求转发回小 AI（携带用户输入值），**系统不解析 `data`，只转发**。

type	控件	触发后转发内容
`button`	按钮	`{action_id, params}`
`submit_form`	表单提交按钮	`{action_id, form_values: {...}}`
`select`	下拉菜单	`{action_id, selected: "..."}`
`checkbox`	复选框	`{action_id, checked: [...]}`
`radio`	单选组	`{action_id, selected: "..."}`
`input`	文本输入框	`{action_id, value: "..."}`
`link`	链接	`{action_id, href}`
`toggle`	开关	`{action_id, on: true/false}`
`slider`	滑块	`{action_id, value: 75}`
`datetime`	日期时间选择器	`{action_id, value: "2026-08-14T22:00"}`

actions 定义示例

```
"actions": [
    {
        "type": "button",
        "label": "播放",
        "action_id": "play_song",
        "params": {"song_id": "s_001"}
    },
    {
        "type": "select",
        "label": "音质",
        "action_id": "set_quality",
        "options": ["标准", "高品", "无损"]
    },
    {
        "type": "submit_form",
        "label": "搜索",
        "action_id": "search",
        "fields": [
            {"name": "keyword", "label": "关键词", "type": "text"}
        ]
    }
]
```

用户点击"播放"按钮后，系统将 `{"action_id": "play_song", "params": {"song_id": "s_001"}}` 转发给小 AI，小 AI 再次规划 `steps` 调用原子工具完成播放，并返回新的 `view_type` + `data` + `actions`。

系统转发 actions 的机制：

1. 每个小AI在 `manifest.json` 中声明自己的 handle 接口路径：

```
{
  "name": "音乐应用",
  "entry": "main.py",
  "handle_endpoint": "/handle"
}
```

1. 系统启动时为每个小AI维护路由表：

```
{
  "音乐应用": {"port": 5001, "endpoint": "/handle"},
  "代码应用": {"port": 5002, "endpoint": "/handle"}
}
```

1. 用户触发 actions 后，系统根据小AI名称查路由表，拼出完整 URL：`http://localhost:{port}{endpoint}`
2. 系统将 action_request POST 到该 URL：

```
{
  "action_id": "play_song",
  "params": {"song_id": "s_001"},
  "session_id": "sess_xxx"
}
```

1. 小AI 收到后，在 `handle` 中通过 `action_request` 参数识别并处理

6.4 steps：工具调用计划

`steps` 是小 AI 规划的工具调用序列。系统执行器按顺序在安全沙箱中执行，将每步结果回传小 AI。每个 step 调用一个原子工具：

```
"steps": [
    {
        "step_id": 1,
        "tool": "http_request",
        "args": {
            "method": "GET",
            "url": "https://api.music.example.com/search",
            "params": {"keyword": "周杰伦"}
        }
    },
    {
        "step_id": 2,
        "tool": "file_write",
        "args": {
            "path": "workspace/recent.json",
            "content": "{{step_1.result}}"
        }
    }
]
```

> `{{step_1.result}}` 表示引用上一步的执行结果，系统执行器负责模板替换。

6.5 完整返回消息示例

```
{
    "status": "ok",
    "view_type": "list",
    "data": {
        "title": "周杰伦的歌曲",
        "items": [
            {"id": "s_001", "title": "七里香", "artist": "周杰伦", "duration": "4:59"},
            {"id": "s_002", "title": "稻香", "artist": "周杰伦", "duration": "3:43"}
        ]
    },
    "actions": [
        {"type": "button", "label": "播放", "action_id": "play", "params": {"song_id":
"s_001"}},
        {"type": "button", "label": "播放", "action_id": "play", "params": {"song_id":
"s_002"}},
        {"type": "input", "label": "筛选", "action_id": "filter"}
    ],
    "message": "为你找到周杰伦的 2 首歌"
}
```

七、原子工具箱（核心 • Beta 2 新增）

7.1 设计理念

系统提供一组**原子工具**，不预设任何高级 API。小 AI 接收用户指令后，自己规划如何组合原子工具完成任务。系统在启动时为小 AI 注入上下文（角色、工作目录、当前状态），小 AI 通过上下文感知环境。

> **硬性约束**：系统不硬编码应用类型（不预设"音乐 API""代码 API"），小 AI 不依赖预设 API。所有高级能力都由小 AI 用原子工具组合实现。

7.2 10 种原子工具

工具名	功能	关键参数
`http_request`	发起 HTTP 请求	`method`, `url`, `headers`, `params`, `body`
`file_read`	读取文件	`path`（相对 workdir）
`file_write`	写入文件	`path`, `content`, `mode`（覆盖/追加）
`file_list`	列出目录	`path`, `pattern`
`shell_exec`	执行系统命令	`cmd`, `timeout`, `cwd`（限定在 workdir）
`process_start`	启动子进程	`cmd`, `args`, `background`
`db_query`	数据库查询	`dsn`, `sql`, `params`
`cache_get`	读取缓存	`key`
`cache_set`	写入缓存	`key`, `value`, `ttl`
`notify`	发送通知	`title`, `body`, `level`

7.3 工具执行安全约束

约束	说明
工作目录限制	`file_*` 与 `shell_exec` 仅限 `workdir` 内，越界请求被拒绝
命令白名单（可选）	`shell_exec` 可在 manifest 中配置允许的命令前缀
超时控制	每个工具有独立超时，超时自动终止
网络白名单（可选）	`http_request` 可配置允许的域名
审计日志	所有工具调用记录到 `tool_calls_log`，可回溯
资源限额	CPU/内存/磁盘单次调用上限

7.4 小 AI 自主组合示例

用户指令："帮我下载周杰伦的七里香到本地"

小 AI 规划的 `steps`：

```
"steps": [
    {
        "step_id": 1,
        "tool": "http_request",
        "args": {
            "method": "GET",
            "url": "https://api.music.example.com/search",
            "params": {"keyword": "七里香 周杰伦"}
        }
    },
    {
        "step_id": 2,
        "tool": "http_request",
        "args": {
            "method": "GET",
            "url": "{{step_1.result.url}}",
            "save_to": "workspace/qilixiang.mp3"
        }
    },
    {
        "step_id": 3,
        "tool": "file_list",
        "args": {"path": "workspace"}
    }
]
```

系统执行器按顺序执行三步，把每步结果回传小 AI，小 AI 最终返回：

```
{
    "status": "ok",
    "view_type": "text",
    "data": {"content": "已下载《七里香》到 workspace/qilixiang.mp3"},
    "actions": [
        {"type": "button", "label": "播放", "action_id": "play_local", "params": {"path":
"workspace/qilixiang.mp3"}}
    ],
    "message": "下载完成"
}
```

> 全程没有调用任何"音乐 API"——小 AI 用 `http_request` + `file_list` 两个原子工具组合完成了任务。

八、SDK 设计（核心 • Beta 2 重构）

8.1 目标

让第三方开发者能用最少的代码编写一个 Triton AIOS 应用（小 AI）。

8.2 SDK 接口规范

开发者继承基类，实现核心方法。Beta 2 SDK **新增原子工具调用接口**，**移除 `render()` 与 `capabilities` 声明**：

```
from aios_sdk import AIOSAgent, tool

class MyAgent(AIOSAgent):
    # 1. 必填：应用元信息
    name = "我的小AI"
    description = "我能做什么"
    icon = "📦"

    # 2. 必填：角色提示词（系统启动时注入上下文）
    role_prompt = "你是一个XXX小AI，负责..."

    # 3. 必填：接收用户指令，返回视图与（可选的）工具调用计划
    def handle(self, instruction, context, action_request=None):
        """
        instruction: 用户自然语言指令
        context: 系统注入的上下文（role/workdir/cwd_state/available_tools/tool_results/...）
        action_request: 用户触发 actions 时的回传（None 表示首次调用）
       返回结构见下方"返回结构说明"
        """
        pass

    # 4. 可选：注册自定义原子工具（扩展工具箱）
    @tool(name="my_custom_tool", desc="自定义工具")
    def my_tool(self, arg1):
        pass
```

返回结构说明：

- `steps`（可选）：工具调用计划，交给系统执行器执行，执行结果会回传到 `context.tool_results`
- `view_type` + `data` + `actions`（必填）：交给桌面渲染器渲染
- 如果 `steps` 还没执行完，可以先返回 `view_type: "loading"` 作为占位视图
- 当 `steps` 执行完成并回传结果后，小AI 应返回完整的最终视图

示例：

```
return {
    "steps": [...],           # 给执行器
    "view_type": "loading",   # 给渲染器（占位）
    "data": {},
    "actions": [],
    "message": "正在处理..."
}
```

context.tool_results 格式：

系统执行器执行完 `steps` 后，会将结果写入 `context.tool_results`，格式如下：

```
{
  "step_1": {
    "tool": "http_request",
    "status": "ok",
    "result": {"songs": [{"id": "s_001", "name": "七里香"}]}
  },
  "step_2": {
    "tool": "file_write",
    "status": "ok",
    "result": {"path": "workspace/recent.json", "size": 1024}
  }
}
```

小AI 在 `handle` 中通过 `context.tool_results` 获取每一步的执行结果，用于组织后续的 `view_type` + `data`。

8.3 原子工具调用接口（新增）

SDK 提供统一的原子工具调用接口，小 AI 在 `handle` 中通过 `self.tools` 调用，或返回 `steps` 由系统执行器执行：

```
class AIOSAgent:
    # 原子工具调用接口（小AI 可直接同步调用）
    def call_tool(self, tool_name, **args):
        """
       同步调用原子工具，立即返回结果。
        tool_name: http_request / file_read / file_write / file_list /
                   shell_exec / process_start / db_query /
                   cache_get / cache_set / notify
        """
        pass

    # 返回工具调用计划（异步，由系统执行器执行）
    def plan_steps(self, steps):
        """
       返回 steps 列表，交给系统执行器在沙箱中执行，
       执行完毕后结果回传 handle 的 context.tool_results。
        """
        pass
```

两种调用模式：

模式	说明	适用场景
同步调用 `call_tool`	小 AI 在 `handle` 内直接调用，立即拿结果	简单、单步、需要即时判断
异步规划 `plan_steps`	小 AI 返回 `steps`，系统执行器执行后回传	复杂、多步、需安全沙箱审计

8.4 最小应用示例

```
from aios_sdk import AIOSAgent

class HelloAgent(AIOSAgent):
    name = "Hello小AI"
    description = "打招呼"
    icon = "▫"
    role_prompt = "你是一个友好的小AI，负责打招呼"

    def handle(self, instruction, context, action_request=None):
        # 用原子工具读取本地问候语
        greeting = self.call_tool("file_read", path="greeting.txt")
        return {
            "status": "ok",
            "view_type": "text",
            "data": {"content": f"{greeting}，{instruction}"},
            "actions": [
                {"type": "button", "label": "再说一次", "action_id": "again"}
            ],
            "message": "已回复"
        }
```

8.5 SDK 自动提供的功能

功能	说明
端口分配	自动监听系统分配的端口
进程管理	自动启动/停止/重启
上下文注入	自动注入 role/workdir/cwd_state/available_tools
工具调度	自动执行 `steps`，回传结果
视图渲染	自动把 `view_type` + `data` + `actions` 交给桌面渲染器
日志记录	自动记录应用运行日志与工具调用审计
错误处理	自动捕获异常并返回标准格式
跨域通信	自动配置 CORS
父窗口通信	自动注入 postMessage 桥接代码（用于 actions 回传）

8.6 应用交互方式

方式	用途
原子工具调用	小 AI 调用系统提供的工具（HTTP/文件/命令等）
actions 回传	用户操作触发 → 系统转发回小 AI `handle` 的 `action_request`
WebSocket	实时推送数据（如播放进度）
postMessage	向桌面发送系统级指令（如移动 UI 元素，见第九章）

> Beta 2 变更 移除 v1.0 的"应用前端 `fetch` 调用自己 `/api/*` "模式。应用不再有独立前端，UI 由系统统一渲染。

九、系统控制应用（特殊应用）

9.1 用途

系统控制应用是一个特殊的系统级小 AI，它可以让用户通过自然语言或点击按钮来修改桌面 UI。

9.2 工作原理

1. 系统控制小 AI 通过 `cache_get` 获取当前桌面的 UI 状态快照
2. 用户通过界面或语音下达修改指令
3. 小 AI 规划 `steps`（用 `shell_exec` 或自定义工具生成 UI 操作指令）
4. 通过 `postMessage` 将操作指令发送给桌面系统
5. 桌面系统执行实际的 DOM 操作

9.3 桌面 UI 状态快照格式

```
{
    "elements": [
        {
            "id": "btn_play",
            "type": "button",
            "label": "播放",
            "x": 100, "y": 200,
            "width": 80, "height": 36,
            "visible": true
        },
        {
            "id": "panel_chat",
            "type": "panel",
            "x": 20, "y": 50,
            "width": 300, "height": 400
        }
    ]
}
```

9.4 支持的 UI 操作指令

指令	说明	参数
move	移动元素	id, x, y
resize	调整大小	id, width, height
hide	隐藏元素	id
show	显示元素	id
change_color	改变颜色	id, color
change_text	改变文字	id, text
rotate	旋转元素	id, degrees

十、语音交互设计

10.1 语音输入

功能	说明
触发方式	点击任务栏 ⌧按钮
识别语言	中文（zh-CN）
识别模式	单次识别（非连续）
识别结果	转文字后自动发送给主控

10.2 语音播报

功能	说明
触发时机	系统回复用户、系统状态提醒
播报内容	小 AI 返回的 `message` 字段
语速	0.9（稍慢，便于理解）
语言	中文（zh-CN）

10.3 语音指令路由

```
语音输入 → 转文字 → 发送给主控 → 主控匹配小AI → 小AI 规划 steps
→ 系统执行器执行 → 小AI 返回视图 → 桌面渲染 → 语音播报 message
```

十一、系统启动流程

第 1 步：系统启动

1. 用户运行 `desktop_app.py`
2. 系统初始化桌面环境（创建 HTML 页面、加载视图渲染器）

第 2 步：扫描应用

1. 扫描 `apps/` 目录
2. 读取每个子目录的 `manifest.json`（含 `role_prompt`、`workdir`）
3. 为每个应用分配空闲端口
4. 记录端口映射到 `port_map.json`

第 3 步：启动应用 + 注入上下文

1. 为每个应用启动独立子进程（传入 `APP_PORT` 环境变量）
2. **为每个小 AI 注入上下文**（角色、工作目录、当前状态、可用工具列表）
3. 各应用在分配的端口上启动服务
4. 系统确认所有应用启动成功

第 4 步：渲染桌面

1. 系统主界面加载
2. 任务栏显示所有已加载应用图标
3. 桌面背景渲染
4. 时钟开始走动

第 5 步：就绪

1. 系统状态显示为"就绪"
2. 等待用户交互（点击应用/语音输入/文字输入）
3. 语音播报"欢迎使用 Triton AIOS"

十二、用户交互流程

12.1 通过点击打开应用

```
用户点击任务栏应用图标
  ↓
桌面检查该应用是否已打开
  ├── 已打开 → 将窗口提到最前
  └── 未打开 → 创建新窗口
                ↓
          系统向小AI 发送空指令（获取初始视图）
                ↓
          小AI 返回 view_type + data + actions
                ↓
          桌面渲染器按 view_type 渲染，按 actions 生成控件
                ↓
          窗口动画弹出
```

12.2 通过语音/文字完成任务

```
用户说："帮我播放周杰伦的歌"
  ↓
主控接收输入，注入上下文，调度音乐小AI
  ↓
音乐小AI 规划 steps:
 ① http_request 搜索周杰伦
 ② cache_set 缓存搜索结果
  ↓
系统执行器执行 steps，结果回传小AI
  ↓
小AI 返回 view_type=list + data(歌单) + actions(播放按钮)
  ↓
桌面渲染器渲染歌单列表与播放按钮
  ↓
用户点击"播放" → 系统转发 action_request 回小AI
  ↓
小AI 再次规划 steps 调用 http_request 播放
  ↓
返回 view_type=media + data(播放地址) + actions(暂停/下一首)
  ↓
语音播报 message
```

12.3 通过语音修改 UI

```
用户说："把播放按钮移到左上角"
  ↓
主控识别为"系统控制"意图，调度系统控制小AI
  ↓
系统控制小AI 规划 steps:
 ① cache_get 获取UI快照
 ② 解析"播放按钮"→ btn_play，"左上角"→ (20,20)
  ↓
小AI 通过 postMessage 向桌面发送 move 指令
  ↓
桌面执行：移动 btn_play 到 (20, 20)
  ↓
小AI 返回 view_type=text + data(操作结果)
  ↓
语音播报："已将播放按钮移到左上角"
```

十三、应用开发模板

13.1 目录模板

```
my_agent/
├── manifest.json      # 含 role_prompt、workdir
├── main.py            # 小AI 入口
├── role.md            # 角色提示词（可选）
├── workspace/         # 小AI 工作沙箱
└── requirements.txt
```

13.2 manifest.json 模板

```
{
    "name": "应用名称",
    "icon": "📦",
    "description": "应用功能描述",
    "entry": "main.py",
    "role_prompt": "你是一个XXX小AI，负责...",
    "workdir": "workspace",
    "author": "你的名字",
    "version": "1.0.0"
}
```

13.3 main.py 模板

```
import os
from flask import Flask, request, jsonify
from aios_sdk import AIOSAgent

PORT = int(os.environ.get("APP_PORT", 5001))

class MyAgent(AIOSAgent):
    name = "我的小AI"
    description = "应用功能描述"
    icon = "📦"
    role_prompt = "你是一个XXX小AI，负责..."

    def handle(self, instruction, context, action_request=None):
        # 1. 小AI 规划工具调用计划
        steps = [
            {
                "step_id": 1,
                "tool": "http_request",
                "args": {"method": "GET", "url": "https://api.example.com/data"}
            }
        ]
        # 2. 交给系统执行器执行（结果会在 context.tool_results 中）
        # 3. 组织返回
        return {
            "status": "ok",
            "view_type": "list",
            "data": {"items": []},
            "actions": [
                {"type": "button", "label": "刷新", "action_id": "refresh"}
            ],
            "steps": steps,
            "message": "已完成"
        }

agent = MyAgent()
app = Flask(__name__)

@app.route('/handle', methods=['POST'])
def handle():
    payload = request.json
    result = agent.handle(
        instruction=payload.get('instruction'),
        context=payload.get('context'),
        action_request=payload.get('action_request')
    )
    return jsonify(result)
```

```
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=PORT, debug=False)
```

13.4 前端通信方式（actions 回传）

用户触发 `actions` 后，系统自动将操作请求 POST 到小 AI 的 `/handle`：

```
{
    "instruction": null,
    "context": {"role": "...", "workdir": "...", "available_tools": [...]},
    "action_request": {
        "action_id": "play_song",
        "params": {"song_id": "s_001"}
    }
}
```

小 AI 在 `handle` 中根据 `action_request` 再次规划 `steps`，返回新的视图。

十四、系统配置文件说明

14.1 config.json

```
{
    "deepseek_api_key": "",
    "default_voice": "zh-CN",
    "voice_rate": 0.9,
    "tool_timeout": {
        "http_request": 30,
        "shell_exec": 60,
        "file_read": 5
    },
    "tool_sandbox": {
        "enable_shell_whitelist": true,
        "shell_whitelist": ["ls", "cat", "echo", "python3"],
        "http_domain_whitelist": []
    }
}
```

14.2 port_map.json

```
{
    "music_app": 5001,
    "code_app": 5002,
    "system_agent": 5100
}
```

十五、开发路线图

阶段	目标	内容
Alpha 1	桌面骨架	窗口管理、任务栏、应用启动
Alpha 2	视图渲染器	10 种 view_type 渲染、10 种 actions 控件生成
Alpha 3	原子工具箱	10 种原子工具实现 + 安全沙箱执行器
Alpha 4	应用 SDK	基础 SDK、上下文注入、工具调用接口
Alpha 5	主控调度	意图理解、小 AI 匹配、steps 转发
Alpha 6	系统控制	UI 修改能力、postMessage 通信
Alpha 7	语音交互	语音识别 + 语音播报
Beta 1	应用市场	多应用生态、应用热加载
Beta 2	系统完善	错误处理、日志、性能优化

十六、总结

Triton AIOS 的核心设计理念：

理念	说明
应用即文件夹	降低开发门槛，即插即用
进程级隔离	保证系统稳定性
统一通信协议	含 `view_type` / `actions` / `steps`，保证可扩展性
AI 只调度不执行	小 AI 规划 steps，系统执行器安全执行
系统提供原子工具，小 AI 自主组合	不预设高级 API，能力由组合涌现
数据不透明	系统不解析 `data`，只渲染与转发
UI 由系统统一渲染	小 AI 只返回 view_type + data + actions
语音原生支持	保证交互自然性

> 这套架构的本质是：**系统是工具提供者与执行器，小 AI 是思考者与规划者，桌面是渲染器。三者各司其职，能力由原子工具的自由组合涌现，而非预设 API 定义。**

文档版本：Beta 2.5　最后更新：2026年8月